

Pestaña 1

Trabajo previo de la clase

Pregunta: ¿Qué sucede con los fallos en **caché** al aplicar vectorización de instrucciones?
¿Se podría pensar que la **vectorización** sin datos alineados puede aumentar la cantidad de fallos?

Pregunta: En el video de técnicas generales de optimización se menciona que hay casos en los cuales los saltos condicionales impiden aplicar el procesamiento **vectorial** en el código y casos en los que no, ¿Cómo se darían ambos casos?

Pregunta: a que se refiere cuando dice que si el compilador no es capaz de **vectorizar** un código debe hacerlo el programador, como se diseña un código vectorizador?

Pregunta: Teniendo en cuenta la diapositiva donde menciona que "Los programas deberían ser contruidos con más de un thread para aprovechar la capacidad completa del microprocesador." (14) **¿Cómo el modelo multicore soluciona la barrera del ILP o simplemente la traslada cambiando el foco al software?**

Pregunta: ¿Cuántos ciclos se necesita para acceder a la memoria?

Pregunta: ¿Multi core es lo mismo que multi cpu? SI

Pregunta: ¿El compilador de C puede generar threads si lo considera necesario y no hay dependencias?

Pregunta: ¿malloc puede acceder a la memoria de otro programa? JAMÁS, MALLOC PIDE MEMORIA PARA UN PROCESO, no hace otra cosa que eso.

Pregunta: ¿Que pasa si en C, se abusa de direcciones de registros?

Relación con otro tema: Dentro de las técnicas generales de optimización de código, la técnica de "**código en línea**" me hizo recordar a una de las buenas prácticas de programación al inicio de la carrera cuando nos indicaban que a la hora de modularizar, cada módulo debía tener sentido de existir y no hacerlo por el simple hecho de modularizar. Que justamente, en caso de que sean pocas líneas en el módulo, que convenía dejarlo dentro del código principal.

Pregunta: En **VLIW** las instrucciones son planificadas estáticamente por el compilador. ¿Qué pasa cuando hay un retraso que no se puede predecir, como un fallo de caché? En situaciones donde pudieran ocurrir mas fallos de cache ,sseria mas conveniente un superescalar?

En multiprocesamiento a nivel de chip, ¿a qué se refiere "replicar la CPU"? ¿cómo se sería a nivel de hardware, qué componentes se comparten y cuáles se duplican? , ¿qué ventajas y limitaciones tiene esta replicación en términos de rendimiento y consumo de energía?

La unidad de encaminamiento del camino de datos funcionaría para solucionar dependencias de datos realizando el reenvío de operandos.

- 1- ILP (Instrucciones) (Alejandro Abele, Jose Quintanilla)
- 2- [TLP \(Hilos\)](#) (Tisiano Arias, Adriano Guido , Torres Andres)
- 3- [DLP \(Datos\)](#) (Facundo Muñoz, Sepulveda Luciano)
- 4- [Técnicas de optimización](#) (Ferraris Facundo, Marisol Pereira)
- 5- Instrucciones vectoriales (Sthefany Cedeño, Victoria Bugli)
- 6- Colaboración con el compilador. (Rodrigo Chavez)

Planilla del oyente

1. Comprensión personal

- ☐ Lo comprendí
- ☐ Tengo dudas

2. Calidad de la exposición (1–4)

Nivel	Qué hacen los estudiantes
1	Repiten contenido del video: Enumeran conceptos o frases casi sin transformación ni explicación propia.
2	Organizan el contenido: Ordenan, resumen o clasifican información.
3	Explican los principios: Identifican la idea central y explican cómo funciona el concepto o cuál es la lógica.
4	Exponen implicancias ("moralejas"): explican para qué sirve conocer el concepto.

Preguntas

Tema ILP:

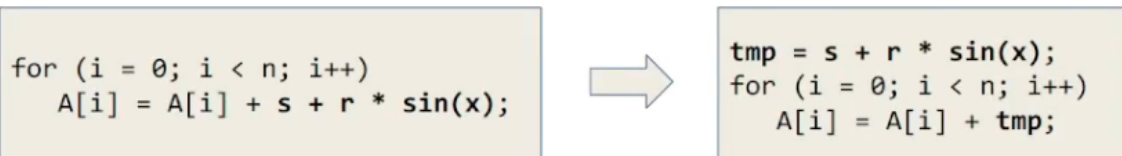
TLP:

Tecnicas de optimización

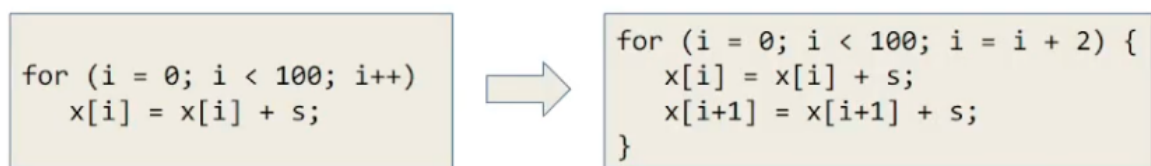
Técnicas de Optimización

Objetivo: hacer menos trabajo y más liviano

- **Extracción de subexpresiones comunes repetitivas:** Evitar hacer cálculos en repetitivas. No siempre lo realiza el compilador en tipo float.



- **Código en línea:** evitar hacer funciones que sean simples. Se evitan saltos para llamada y retorno, la copia de datos de entrada a registros, accesos a memoria.
- **Desenrollado de bucles:** reducir la cantidad de iteraciones en bucles de forma balanceada para no aumentar de forma excesiva el uso de registros.



- **Evitación de saltos condicionales:** reducir dependencias de control (por ejemplo los saltos de bucles impide el procesamiento vectorial).

pwer

- **Código amigable con la caché:** alinear los datos de tal modo que el acceso a ellos tenga la menor cantidad de fallos de caché. Por ejemplo, en C se utiliza `int posix_memalign` para solicitud de memoria dinámica alineada, e `int x __attribute__((aligned(N)))` para alinear a `x` de forma estática `N` bytes.

```
int posix_memalign(void **memptr, size_t alignment, size_t size);
```

```
int x __attribute__((aligned(N)))
```

Data Level Parallelism

Data Level Parallelism (DLP)

DLP busca aplicar las mismas instrucciones a múltiples datos, utilizando un modelo SIMD en la taxonomía de Flynn, para lograrlo existen dos tipos de procesadores:

- **Procesador vectorial.** Un único elemento de procesamiento implementa un pipeline aritmético para aplicar operaciones sobre distintos datos de un mismo vector en simultáneo.
- **Procesador matricial.** La unidad de control envía las mismas instrucciones a distintos elementos de procesamiento. Es importante distinguir que estos elementos de procesamiento son simples.

Para que DLP funcione no debe existir dependencia de instrucciones, ya que rompe el paralelismo.

Ejemplo

```
for (i = 0; i < 4; i++)  
    A[i] = B[i] + C[i];
```

Datos:

B = [1, 2, 3, 4]

C = [10, 20, 30, 40]

Resultado esperado:

A = [11, 22, 33, 44]

Procesadores matriciales (array processors)

La unidad de control envía la misma instrucción a muchos elementos de procesamiento en paralelo, cada elemento de procesamiento toma un elemento distinto:

PE1 $\rightarrow A[0] = B[0] + C[0] \rightarrow 1 + 10 = 11$

PE2 $\rightarrow A[1] = B[1] + C[1] \rightarrow 2 + 20 = 22$

PE3 $\rightarrow A[2] = B[2] + C[2] \rightarrow 3 + 30 = 33$

PE4 $\rightarrow A[3] = B[3] + C[3] \rightarrow 4 + 40 = 44$

Es más eficiente pero más costoso.

Procesadores vectoriales (vector processors)

Cada **operación** se separa en otras operaciones aritméticas distintas sobre los registros (por ejemplo alineamiento, suma, normalización, redondeo, etc.), de forma que mientras una de las sumas se está redondeando, la siguiente ya puede "entrar a normalizar". Notemos que esto se implementa de forma física dentro de las unidades funcionales. Es más barato respecto a los circuitos.

Cómo trabaja internamente (pipeline)

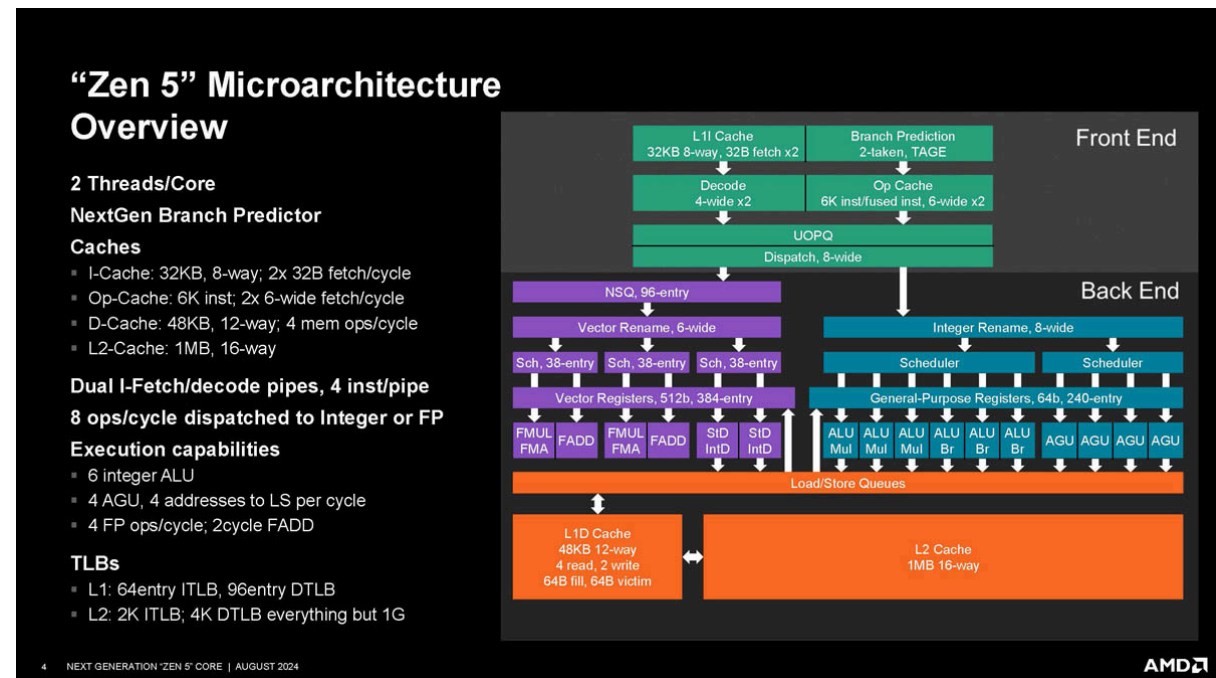
Un procesador vectorial no hace todo junto, sino en etapas superpuestas:

Tiempo →

Elemento 0	Cargar Sumar Guardar
Elemento 1	Cargar Sumar Guardar
Elemento 2	Cargar Sumar Guardar
Elemento 3	Cargar Sumar Guardar

En forma de "gráfico"

Ciclo 1: [Load B0,C0]
Ciclo 2: [Add B0+C0] [Load B1,C1]
Ciclo 3: [Store A0] [Add B1+C1] [Load B2,C2]
Ciclo 4: [Store A1] [Add B2+C2] [Load B3,C3]
Ciclo 5: [Store A2] [Add B3+C3]
Ciclo 6: [Store A3]



TLP

Paralelismo de Nivel de Hilos

Se refiere a realizar distintas tareas al mismo tiempo gracias a varios núcleos, por ejemplo, varios cocineros, uno hace asado, otro pasteles, etc.

También lo vemos de manera cotidiana cuando usamos la computadora, usualmente usamos spotify, google, etc, todo al mismo tiempo.

Dentro de las técnicas para explotar el paralelismo a nivel de hilos tenemos.

- **Multithreading:** En este caso se tiene un solo núcleo, pero el hardware está diseñado para mantener el estado (registros, y contador de programa) de varios hilos a la vez, a esto se le llama cambio de contexto.

El procesador va alternando en el tiempo. En un ciclo de reloj o bloque de tiempo, ejecuta las instrucciones del Hilo 1 y en el siguiente ciclo ejecuta las instrucciones del Hilo 2. También funciona para ocultar la latencia, ya que si un hilo necesita un dato de la Ram, este es un viaje bastante largo y el procesador se quedaría sin hacer nada hasta obtener ese dato de la ram, pero gracias a esta técnica, otro hilo puede aprovechar el procesador para ejecutar sus **instrucciones**, es decir el procesador puede ir alternando de hilo en hilo mientras sea necesario. Si el hilo 1 necesita un dato de la Ram, automáticamente se realiza un cambio de contexto y se accede al contador del hilo 2 y su banco de registros y este hilo empieza a ejecutarse, hasta que el hilo 1 obtenga el dato de la Ram. Gracias a este diseño a nivel físico el cambio de contexto resulta en un ciclo reloj, lo cual hace que esta técnica sea muy rápida e imperceptible en los cambios de hilos.

- **Multithreading simultáneo:** El multithreading simultáneo es similar al anterior, solo que permite que las instrucciones a ejecutar en un mismo ciclo de reloj pertenezcan a hilos distintos. Se logra sin grandes cambios en la arquitectura básica del procesador, debiéndose resolver el hecho de contener datos de múltiples hilos. Cada núcleo del procesador admite cierta cantidad de hilos, **usualmente 2**. Desde el punto de vista del Sistema Operativo, un procesador con multithreading simultáneo aumenta la cantidad de unidades lógicas de procesamiento. Por ejemplo, un procesador con 2 núcleos y con capacidad para ejecutar dos threads simultáneamente presenta 4 unidades lógicas de procesamiento, que se las denomina CORE LÓGICOS, para distinguirlas de los core físicos.

Ejemplo comparativo entre Multithreading y Multithreading simultáneo:

Multithreading

IF	ID	EX	MEM	WB		
IF	ID	EX	MEM	WB		
	IF	ID	EX	MEM	WB	
	IF	ID	EX	MEM	WB	
		IF	ID	EX	MEM	WB
		IF	ID	EX	MEM	WB

Multithreading simultáneo

IF	ID	EX	MEM	WB		
IF	ID	EX	MEM	WB		
	IF	ID	EX	MEM	WB	
	IF	ID	EX	MEM	WB	
		IF	ID	EX	MEM	WB
		IF	ID	EX	MEM	WB

En la parte izquierda de la imagen se puede apreciar como la ejecución de instrucciones en un mismo ciclo de reloj corresponde a un único hilo, a diferencia de la parte derecha, donde es posible que instrucciones de distintos hilos se ejecuten en un mismo ciclo.

El multithreading simultáneo representa una ventaja sobre el multithreading en el hecho de poder aprovechar mejor los recursos del sistema, al ocupar la totalidad del pipeline. Supongamos, por ejemplo, que la cantidad de instrucciones por comenzar a ejecutar pertenecientes a un hilo sea inferior a la cantidad que es posible ejecutar en paralelo por etapa. En el caso de multithreading quedaría parte del pipeline sin utilizar, en cambio, en el multithreading simultáneo se podría aprovechar esto para ejecutar alguna instrucción de otro hilo que esté en espera para comenzar su ejecución, evitando situaciones puntuales de tiempo ocioso.

- **Multiprocesamiento de nivel de chip:** En esta técnica, se aumenta la cantidad de núcleos en cada procesador para que cada uno ejecute un hilo propio. A diferencia del multithreading cuya función es ejecutar varios hilos por vez dentro de un solo núcleo (alternando el tiempo entre ellos), en esta técnica un procesador tiene varios núcleos físicos donde cada uno ejecuta un hilo diferente (todos al mismo tiempo). Esto permite un paralelismo real y no una simulación por cambio de contexto.

Ejemplo: En el caso hipotético de que se tenga que lavar, secar y doblar ropa constantemente, teniendo un solo núcleo, una sola persona (hilo) tendría que hacer las 3 actividades. En cambio si tenemos 3 núcleos donde hay una persona (hilo) en cada uno, uno se encargaría de lavar, otro de secar y el último de doblar.

Instrucciones Vectoriales

Instrucciones Vectoriales

Las instrucciones vectoriales son comandos de procesador que permiten operar sobre múltiples datos simultáneamente ([SIMD](#) - Single Instruction, Multiple Data) en lugar de uno por uno

Son instrucciones que se aplican sobre vectores, mientras que las instrucciones escalares operan sobre un solo dato por vez, las instrucciones vectoriales operan sobre varios datos al mismo tiempo, en paralelo.

Aumenta el rendimiento contra instrucciones escalares

Se implementan en procesadores de propósito general utilizando la técnica de pipeline (dividir una operación en varias etapas y ejecutarlas en forma superpuesta)

Instrucción vectorial				Instrucción escalar
A3	A2	A1	A0	A
B3	B2	B1	B0	+
				B
-----				----
A3+B3	A2+B2	A1+B1	A0+B0	A+B

En la [Vectorización Automática](#) el compilador convierte instrucciones escalares a vectoriales, en algunos casos triviales o con ayuda del programador en otros casos.

Las [Funciones intrínsecas](#) se usan cuando el compilador no logra vectorizar solo. El programador escribe un código que puede traducirse casi directamente a instrucciones del procesador.

Resumiendo..

- ★ Los registros vectoriales son más grandes (no de a uno como los escalares), permiten procesar varios datos a la vez.
- ★ Mientras más chicos los datos, más operaciones entran en paralelo.
- ★ Las instrucciones varían según tamaño de datos y tipo de operación.